

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_471a9cdd-d1d\agent-abd701c549ab9ea49.jsonl`  
- SHA-256: `be451c975ad3488ec2597046f709c967c4bad3ed8033a718042e057b36e3e69c`  
- Source modified: `2026-06-10T09:32:13+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_471a9cdd-d1d`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T09:28:10.498Z)

CONTEXT ? you are advising a real architecture decision for a solo-founder project (owner = avidullu).

Repos on disk (read-only; cite file paths for every claim):

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? FastAPI+SQLite+ffmpeg rally/highlight pipeline. Pluggable registries (segmenters/providers/validators/sports/annotations). Key seams: backend/pipeline/segmenters/base.py (registry), backend/pipeline/detectors/base.py (DetectorRunner ABC ? WASB/TrackNet run as WSL subprocesses from an EXTERNAL repo ~/models/WASB-SBDT), backend/providers/ (Gemini = external API). Eval stack backend/eval/ (metrics.match_intervals = the scoring spine, rally_seq_proto.py = the learned Gen-0 prototype seed, eval_partial_labels, calibrate_local LOVO, regression gate w/ tracked baseline in eval_baselines/). CI = .github/workflows/ci.yml (an import-smoke job WITHOUT the GPU stack + a full-suite job with CPU torch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? separate repo, the data system-of-record (golden labels, licensing gate, CVAT ingestion, 'curate export' manifest consumed by the indexer's batch eval).
- Also exists: private sports-obsreport repo = a shared report library consumed as a SOFT dependency by both repos (a working precedent for 'separate repo + consumed as dep').

THE DECISION: the owner is about to greenlight M0.4 (productionize rally_seq_proto into a Gen-0 TAL train?eval?ship-gate harness over ActionFormer/ASFormer (MIT), later Gen-2 = Qwen3-VL fine-tuned via ms-swift) and M0.1 (corpus spine + event-index contract, authored in the collector). QUESTION: should the model/training work live IN the indexer repo, or in a SEPARATE repo whose output is consumed by the indexer as just another segmenter/runner (like WASB and Gemini)? The owner PREFERS isolation ("iterate independently", cites the collector split as precedent) but asked for honest thoughts.

KNOWN FACTS to weigh (verified in a deep audit this week ? do not re-litigate, build on them):

- Cross-repo contract drift IS a real observed cost here: the collector's export header (start,end) vs the indexer's calibrate_wasb loader (start_time,end_time) forked into two incompatible golden-CSV conventions; tuned constants (TUNED/BASELINE) were duplicated-by-copy across files and drifted.
- Import-graph hygiene is a live concern: importing backend.main pulls torch/cv2 via the segmenter registration manifest (lazy-imports were deferred to the async-job slice).
- Authoritative plan docs: docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md ('envision now, build later' ? design every deferred seam up front; train EXPORT-CLEAN ops; on-device later = exported ONNX/CoreML/GGUF artifact + thin shell; the moat = consented dataset + eval harness, NOT the weights), docs/NEXT_STEPS.md, docs/PLATFORM_ARCHITECTURE.md.
- Guardrails: MIT/Apache-2.0/BSD only for code+data+weights deps; trained weights themselves = private/proprietary asset; minors excluded from training; per-user training opt-in; consent ledger planned in M0.1 schema.
- Quality state: learned LOVO mean 0.626 (n=6) vs heuristic 0.480; ship-gate F1@tIoU target still undefined.

- A repo RENAME off 'badminton' is already an owner action item ? repo topology is in flux anyway.
- Training runs on the owner's Windows box via WSL2 + RTX 2070 Super (8GB); Gen-2 may use rented cloud GPUs (Modal/RunPod per the study).
- Solo developer + AI agents do the work; there are ALREADY 4-5 repos (indexer, collector, rally-annotator, sports-obsreport).

RULES: read-only. Your final structured output is consumed programmatically.

ROLE: argue the STRONGEST case for a SEPARATE model/training repo whose output the indexer consumes as another segmenter/runner. Read first: backend/pipeline/detectors/base.py (the ABC's own docs on adding runners), backend/pipeline/segmenters/base.py + wasb_hybrid.py (how an external substrate is consumed today), backend/eval/rally_seq_proto.py (what would move), docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md (export-clean ops, artifact boundary, M0.4), .github/workflows/ci.yml + requirements.txt (dependency weight). Make the case concrete: propose the new repo's scope/name, its deliverable (artifact format + manifest contents), the thin indexer-side runner, where weights live (git-lfs? releases?), how CI works in each repo, and how this interacts with the pending repo rename. Be honest in 'risks' about cross-repo drift (the observed header fork!) and solo-dev overhead.

2. user (2026-06-10T09:28:17.805Z)

```
1      """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3      This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4      of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5      the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7      The ABC lets us:
8      - Keep the current WSL adapters working unchanged for Windows users.
9      - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10     the hybrid segmenters or the model-agnostic windowing brain.
11     - Provide a single place to evolve the healthcheck / error contract and thread
12     detector readiness into per-video reports.
13
14     Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15     - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16       Run detector, return the Windows-side path to the produced trajectory CSV
17       (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18       The caller (hybrid) is responsible for output dir; impls may stage the video
19       into WSL fs for perf.
20
21     - healthcheck() -> tuple[bool, str]
22       Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23       Message is logged (and can be surfaced to reports or UX in future).
24       Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26     Existing behavior is 100% preserved. The only additions are the inheritance and
27     the explicit common type for future implementers.
28
29     How to add a new runner (for docs / future contributors):
30     1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31     2. Implement run_predict and healthcheck (return (bool, str) tuple).
32     3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33       TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34       deliberately static and model-agnostic). If the new detector needs different
35       windowing, it can implement its own or we generalize later.
36     4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37       registry never pulls torch/tf on a machine that doesn't have them).
38     5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39       DetectorRunner instances (they already do the right thing via duck-typing + the
40       type hints we add here).
41     6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42       returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43     7. No change to eval/calibration code paths (they consume the CSV + the static
44       windowing helpers directly).
```

```

46 Optional Linux-native path (the main de-risk goal):
47 A LinuxNativeRunner (or ONNXRunner) would:
48 - Skip all wsl / _stage / conda activate.
49 - Load weights locally (torch or onnx) or call a native binary.
50 - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51 - healthcheck can just check "weights file exists and torch importable + CUDA?".
52 This removes the WSL requirement for Linux users / future SaaS workers while
53 keeping the exact same candidate quality characteristics (or better, once native
54 weights are available).
55
56 Error surfacing into reports:
57 The healthcheck message is already logged by the hybrids on failure. The reporting
58 harvest sees the outcome as "0 candidates" + detector name. When we wire richer
59 segmenter Results (or pass detector_health into emit_indexer_report context), the
60 (ok, msg) can be recorded explicitly under segmentation.details or a new detector
61 stage. The ABC makes that future change local to one place.
62
63 See also:
64 - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
65 - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
66 - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
67 ""
68
69 from abc import ABC, abstractmethod
70 from typing import Optional, Tuple
71
72
73 class DetectorRunner(ABC):
74     """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""
75
76     @abstractmethod
77     def run_predict(self, video_win_path: str, output_win_dir: str,
78                    video_id: Optional[str] = None) -> Optional[str]:
79         """Run inference and return path to trajectory CSV (or None on failure).
80
81         ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
82         videos that happen to share a filename cannot reuse each other's frames or CSV.
83         It is optional for back-compat; callers in the live pipeline always pass it.
84         """
85         pass
86
87     @abstractmethod
88     def healthcheck(self) -> Tuple[bool, str]:
89         """Quick check if the runner (env, weights, etc.) is usable.
90
91         Returns (ok, message). Never raises. Cheap to call before a long run.
92         """
93         pass
94

```

3. user (2026-06-10T09:28:17.889Z)

```

1     from abc import ABC, abstractmethod
2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod

```

```

13     def name(self) -> str:
14         """Unique identifier for this segmenter."""
15         pass
16
17     @property
18     @abstractmethod
19     def description(self) -> str:
20         """Human-readable explanation of this segmenter."""
21         pass
22
23     @abstractmethod
24     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25         """
26         Processes a video to detect play segments, populates SQLite database,
27         and returns a Result.
28
29         Use isinstance(result, Failure) (or result.is_success()) to distinguish
30         "no segments found" from "something went wrong" (missing key, healthcheck
31         failure, etc.). This is the start of the standardized error/result contract.
32         """
33         pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40             """Registers a new segmenter class under its name property."""
41             try:
42                 temp_inst = segmenter_cls(None, {})
43                 name = temp_inst.name
44             except Exception as e:
45                 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}")
46             self._segmenters[name] = segmenter_cls
47
48         def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49             return self._segmenters.get(name)
50
51         def list_available(self) -> Dict[str, str]:
52             """Returns a dict of registered segmenter names and their descriptions."""
53             res = {}
54             for name, cls in self._segmenters.items():
55                 try:
56                     temp_inst = cls(None, {})
57                     desc = temp_inst.description
58                 except Exception:
59                     desc = "No description available."
60                 res[name] = desc
61             return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

4. user (2026-06-10T09:28:18.780Z)

```

1     """PROTOTYPE: a learned per-FRAME rally segmenter (vs the unsupervised windowing
heuristic).
2
3     Motivation (2026-06-08 finding): WASB tracks the shuttle well on Kushagra (97% of GT
rallies contain >=2 net-crossings) yet NO velocity-windowing config segments it
4     (per-video F1 ceiling 0.163), while it works on Adarsh (0.73). The rally SIGNAL is in
5     the trajectory; the hand-tuned heuristic just can't extract it across footage. This
6     prototype tests the hypothesis that a LEARNED model on per-frame trajectory features
7

```

```

8     recovers the rally segments ? including on the footage the heuristic fails on.
9
10    Approach (deliberately small, dependency-light: numpy + scipy + the repo's pure-python
11    LogisticRegression ? no torch/sklearn):
12        1. Per-frame features over a local window from the WASB trajectory: visibility rate,
13           mean/max shuttle speed (frame-width-normalized), net-crossing RATE, x/y spread.
14           All are scale/camera-robust RATES (unlike a raw velocity threshold).
15        2. Per-frame label = inside a human GT rally or not.
16        3. LEAVE-ONE-VIDEO-OUT: train the classifier on the other match(es), test on the
17           held-out one (the honest cross-footage number; no within-video leakage).
18        4. Smooth probabilities -> threshold (tuned on TRAIN) -> contiguous in-rally segments
19           -> merge tiny gaps / drop short -> score vs GT at IoU>=0.5 (same metric as the
20    heuristic).
21
22    This is a directional PROOF-OF-SIGNAL, not a production model (n=2 videos, linear model,
23    hand features). It exists to answer one question: is the rally boundary learnable from
24    the trajectory where the heuristic fails? Compare its held-out F1 to the heuristic LOVO.
25
26    Run:
27    python -m backend.eval.rally_seq_proto --manifest output/human_lovo_manifest.json
28
29    from __future__ import annotations
30
31    import json
32    import argparse
33    from typing import Dict, List, Tuple
34
35    import numpy as np
36    from scipy.ndimage import uniform_filter1d, maximum_filter1d
37    from scipy.stats import rankdata
38
39    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
40    from backend.eval.classifier import LogisticRegression
41    from backend.eval.calibrate_wasb import load_golden_gts
42    from backend.eval.metrics import score
43
44    FEAT_NAMES = ["vis_rate", "spd_mean", "spd_max", "cross_rate", "x_std", "y_std"]
45    Interval = Tuple[float, float]
46
47    def _spread(a: np.ndarray) -> float:
48        return float(np.percentile(a, 95) - np.percentile(a, 5)) if a.size > 1 else 0.0
49
50
51    def build_frame_arrays(points, frame_width: float, fps: float = 30.0) -> Dict:
52        """Per-frame trajectory arrays + per-visible-frame speed and net-crossing events.
53
54        Speed is normalized to fraction-of-frame-width PER SECOND (`* fps`), matching the
55        heuristic windowing brain ? previously it was per-frame, so the same physical
56        shuttle
57        speed read ~2x smaller at 59.94 fps than at 30 fps, confounding cross-footage
58        transfer
59        on a corpus that mixes frame rates. Speed/crossings are also not computed across a
60        long
61        occlusion gap (mirrors the heuristic resetting on invisibility)."""
62        pts = sorted(points, key=lambda p: p.frame)
63        N = (pts[-1].frame + 1) if pts else 1
64        vis = np.zeros(N); x = np.zeros(N); y = np.zeros(N)
65        for p in pts:
66            if 0 <= p.frame < N and p.visible and p.x >= 0:
67                vis[p.frame] = 1.0; x[p.frame] = float(p.x); y[p.frame] = float(p.y)
68
69        mask = vis > 0
70        play_is_x = _spread(x[mask]) >= _spread(y[mask])
71        coord = x if play_is_x else y

```

```

69     net = float(np.median(coord[mask])) if mask.any() else 0.0
70     deadband = 0.06 * (_spread(coord[mask]) if mask.sum() > 1 else 1.0)
71
72     speed = np.zeros(N); spd_mask = np.zeros(N); cross = np.zeros(N)
73     max_gap = max(2, round(0.25 * fps)) # frames; a longer gap = occlusion break, not
motion
74     prev = None
75     for p in pts:
76         f = p.frame
77         if not (0 <= f < N and vis[f]):
78             continue
79         if prev is not None:
80             df = f - prev
81             if 0 < df <= max_gap:
82                 speed[f] = float(np.hypot(x[f] - x[prev], y[f] - y[prev]) / frame_width
/ df * fps)
83                 spd_mask[f] = 1.0
84                 c, pc = coord[f] - net, coord[prev] - net
85                 if abs(c) > deadband and abs(pc) > deadband and (c > 0) != (pc > 0):
86                     cross[f] = 1.0
87             prev = f
88     return dict(N=N, vis=vis, x=x, y=y, speed=speed, spd_mask=spd_mask, cross=cross,
fw=frame_width)
89
90
91     def features(arr: Dict, fps: float, win_sec: float = 0.75, stride_sec: float = 0.1):
92         """Windowed per-frame features sampled at a stride -> (X[n_samples,6],
times[n_samples])."""
93         N = arr["N"]; fw = arr["fw"]
94         size = max(3, int(win_sec * fps) * 2 + 1)
95         vis, speed, spd_mask, cross, x, y = (arr[k] for k in ("vis", "speed", "spd_mask",
"cross", "x", "y"))
96
97         n = uniform_filter1d(vis, size, mode="constant") # mean
visibility (== vis_rate)
98         sm = uniform_filter1d(spd_mask, size, mode="constant")
99         spd_mean = uniform_filter1d(speed, size, mode="constant") / np.maximum(sm, 1e-9)
100         spd_max = maximum_filter1d(speed, size, mode="constant")
101         cross_rate = uniform_filter1d(cross, size, mode="constant") * fps # crossings
/ second
102         xm = uniform_filter1d(x * vis, size, mode="constant") / np.maximum(n, 1e-9)
103         x2 = uniform_filter1d((x * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
104         x_std = np.sqrt(np.maximum(x2 - xm ** 2, 0.0)) / fw
105         ym = uniform_filter1d(y * vis, size, mode="constant") / np.maximum(n, 1e-9)
106         y2 = uniform_filter1d((y * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
107         y_std = np.sqrt(np.maximum(y2 - ym ** 2, 0.0)) / fw
108
109         idx = np.arange(0, N, max(1, int(stride_sec * fps)))
110         X = np.stack([n[idx], spd_mean[idx], spd_max[idx], cross_rate[idx], x_std[idx],
y_std[idx]], axis=1)
111         return np.nan_to_num(X), idx / fps
112
113
114     def labels_for(times: np.ndarray, gts: List[Interval]) -> np.ndarray:
115         y = np.zeros(len(times))
116         for i, t in enumerate(times):
117             for s, e in gts:
118                 if s <= t <= e:
119                     y[i] = 1.0
120                     break
121         return y
122
123
124     def roc_auc(y: np.ndarray, p: np.ndarray) -> float:
125         """Rank-based AUC (Mann-Whitney) with MIDRANK tie handling.

```

```

126
127     Uses scipy.stats.rankdata (average method) so tied probabilities get averaged ranks
128     ?
129     without this, an all-tied classifier scored a spurious order-dependent 0.25..0.75
130     instead of 0.5, and dead-time frames produce identical all-zero feature rows
131     (guaranteed
132     ties), so the headline AUC carried an unquantified bias.""
133     pos, neg = p[y > 0.5], p[y <= 0.5]
134     if pos.size == 0 or neg.size == 0:
135         return float("nan")
136     ranks = rankdata(p) # midranks (ties averaged)
137     return float((ranks[y > 0.5].sum() - pos.size * (pos.size + 1) / 2) / (pos.size *
138     neg.size))
139
140 def probs_to_segments(probs: np.ndarray, times: np.ndarray, threshold: float,
141     merge_gap: float = 1.0, min_dur: float = 1.0, smooth: int = 7) ->
142     List[Interval]:
143     p = uniform_filter1d(probs, max(1, smooth), mode="nearest")
144     on = p >= threshold
145     runs: List[Interval] = []
146     i, n = 0, len(on)
147     while i < n:
148         if on[i]:
149             j = i
150             while j + 1 < n and on[j + 1]:
151                 j += 1
152             runs.append((float(times[i]), float(times[j])))
153             i = j + 1
154         else:
155             i += 1
156     merged: List[Interval] = []
157     for s, e in runs:
158         if merged and s - merged[-1][1] <= merge_gap:
159             merged[-1] = (merged[-1][0], e)
160         else:
161             merged.append((s, e))
162     return [(s, e) for s, e in merged if e - s >= min_dur]
163
164 def _seg_f1(clf, vids: List[Dict], thr: float) -> float:
165     f1s = []
166     for v in vids:
167         segs = probs_to_segments(clf.predict_proba(v["X"]), v["times"], thr)
168         f1s.append(score(segs, v["gts"], 0.5).f1)
169     return sum(f1s) / max(len(f1s), 1)
170
171 def run(specs: List[Dict]) -> dict:
172     data = {}
173     for s in specs:
174         pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
175         arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
176         X, times = features(arr, float(s["fps"]))
177         gts = load_golden_gts(s["labels"])
178         data[s["name"]] = {"X": X, "times": times, "y": labels_for(times, gts), "gts":
179         gts}
180     names = list(data)
181     print(f"=== Learned per-frame rally segmenter ? LEAVE-ONE-VIDEO-OUT ({len(names)}
182     videos) ===")
183     print(f"features: {FEAT_NAMES}\n")
184     results = []
185     for test in names:
186         train = [data[n] for n in names if n != test]

```

```

185         Xtr = np.vstack([v["X"] for v in train]); ytr = np.concatenate([v["y"] for v in
train])
186         clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
187         clf.fit(Xtr, ytr, feature_names=FEAT_NAMES)
188         # tune the decision threshold on TRAIN segment-F1 (no test leakage)
189         thr = max(np.arange(0.2, 0.81, 0.05), key=lambda t: _seg_f1(clf, train,
float(t)))
190         v = data[test]
191         probs = clf.predict_proba(v["X"])
192         auc = roc_auc(v["y"], probs)
193         segs = probs_to_segments(probs, v["times"], float(thr))
194         sc = score(segs, v["gts"], 0.5)
195         # Oracle threshold (fit-on-TEST, upper bound): isolates the representation
ceiling
196         # from the threshold-transfer gap. Optimistic ? for diagnosis only.
197         othr = max(np.arange(0.2, 0.81, 0.05),
198                   key=lambda t: score(probs_to_segments(probs, v["times"], float(t)),
v["gts"], 0.5).f1)
199         osc = score(probs_to_segments(probs, v["times"], float(othr)), v["gts"], 0.5)
200         results.append({"video": test, "thr": float(thr), "auc": auc,
201                       "f1": sc.f1, "precision": sc.precision, "recall": sc.recall,
202                       "n_pred": len(segs), "n_gt": len(v["gts"]),
203                       "oracle_thr": float(othr), "oracle_f1": osc.f1})
204         print(f"[held-out {test}] per-frame AUC {auc:.3f} | LOVO-threshold F1
{sc.f1:.3f} "
205               f"(P {sc.precision:.3f} R {sc.recall:.3f}, thr {thr:.2f} from train) | "
206               f"oracle-threshold F1 {osc.f1:.3f} (thr {othr:.2f}, fit-on-test upper
bound)")
207         mean_f1 = sum(r["f1"] for r in results) / len(results)
208         print(f"\nlearned LOVO mean held-out F1: {mean_f1:.3f}")
209         return {"per_video": results, "mean_f1": mean_f1}
210
211
212     def main() -> None:
213         ap = argparse.ArgumentParser(description=__doc__)
214         ap.add_argument("--manifest", required=True, help="JSON:
[{name,trajectory,fps,frame_width,labels}, ...]")
215         args = ap.parse_args()
216         with open(args.manifest, encoding="utf-8") as f:
217             run(json.load(f))
218
219
220     if __name__ == "__main__":
221         main()
222

```

5. user (2026-06-10T09:28:19.255Z)

```

1     name: CI
2
3     on:
4         push:
5             branches: [master]
6         pull_request:
7
8     permissions:
9         contents: read
10
11     concurrency:
12         group: ci-`${{ github.ref }}
13         cancel-in-progress: true
14
15     jobs:
16         # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
17         # module-level import error shipping while the suite stays green.

```

```

18     # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
19     # on machines without the GPU stack (test_import_smoke stubs what's absent).
20     import-smoke:
21         name: Syntax sweep + import smoke (no GPU stack)
22         runs-on: ubuntu-latest
23         steps:
24             - uses: actions/checkout@v4
25             - uses: actions/setup-python@v5
26               with:
27                 python-version: "3.13"
28             - name: Install minimal deps
29               run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
30             - name: Run import smoke tests
31               run: python -m pytest tests/test_import_smoke.py -v
32
33     full-suite:
34         name: Full test suite (offline, mocked)
35         runs-on: ubuntu-latest
36         steps:
37             - uses: actions/checkout@v4
38             - uses: actions/setup-python@v5
39               with:
40                 python-version: "3.13"
41                 cache: pip
42             - name: System libs for opencv-python
43               run: |
44                 sudo apt-get update
45                 sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
46             - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
47               run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
48             - name: Install requirements
49               run: python -m pip install -r requirements.txt pytest httpx # httpx: fastapi
TestClient
50         # obsreport (private, soft dep) is absent here: reporting tests skip via
51         # pytest.importorskip ? run_all_tests.py prints which.
52         - name: Run full suite
53           run: python run_all_tests.py
54

```

6. user (2026-06-10T09:28:19.613Z)

```

1     fastapi>=0.111.0
2     uvicorn>=0.30.1
3     opencv-python>=4.9.0.80
4     numpy>=1.26.4
5     pydantic>=2.7.1
6     jinja2>=3.1.4
7     python-dotenv>=1.0.0
8     google-genai>=2.4.0
9     torch>=2.12.0
10    torchvision>=0.27.0
11    supervision>=0.21.0,<0.30.0
12
13    # Per-video observability reports (SOFT dependency: the pipeline degrades to a
14    # no-op if absent). Shared, permissively-licensed report toolkit. Pin a tag once
15    # the private repo is published, e.g.:
16    #   obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3
17    # For local development install it editable from a sibling checkout:
18    #   pip install -e ../sports-obsreport --no-build-isolation
19

```

7. user (2026-06-10T09:28:27.761Z)

```

1      """WASB-based hybrid play segmenter.
2
3      Identical three-phase shape to TrackNetHybridSegmenter, but Phase 2 uses **WASB**
4      (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via WSL2
5      (backend/detectors/wasb_runner.py) instead of TrackNetV4. WASB ships pretrained
6      badminton weights and runs today, so this is the live shuttle-motion substrate.
7      Phase 3 (AI semantic boundaries) and Phase 4 (DB) are shared with the other hybrids.
8
9      For the per-video *calibration* goal we mainly use Phase 2 (trajectory -> candidate
10     windows); the windowing thresholds (velocity_thresh / merge_gap / min_window_duration)
11     are what get tuned against golden rally labels.
12     """
13     import os
14     import time
15     import logging
16     import concurrent.futures
17     from typing import List, Dict, Any
18
19     import cv2
20
21     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
22     from backend.utils.video import get_video_duration, extract_video_chunk
23     from backend.sports import sport_registry
24     from backend.providers import provider_registry
25     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
26     from backend.pipeline.detectors.base import DetectorRunner # ABC for the de-risked
detector seam
27
28     logger = logging.getLogger(__name__)
29
30
31     def _fmt_ts(seconds: float) -> str:
32         seconds = max(0, int(seconds))
33         h, rem = divmod(seconds, 3600)
34         m, s = divmod(rem, 60)
35         return f"{h:02d}:{m:02d}:{s:02d}"
36
37
38     class WasbHybridSegmenter(BasePlaySegmenter):
39         @property
40         def name(self) -> str:
41             return "wasb_hybrid"
42
43         @property
44         def description(self) -> str:
45             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
46                     "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
47
48         @staticmethod
49         def _probe_fps_width(video_path: str) -> tuple:
50             cap = cv2.VideoCapture(video_path)
51             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
52             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
53             cap.release()
54             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
55
56         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None,
57                         max_frames: int = None) -> List[Dict[str, Any]]:
58             sport_name = self.config.get("sport", "badminton")
59             try:
60                 sport_profile = sport_registry.get(sport_name)
61             except KeyError as e:

```

```

62         logger.error(str(e))
63         return []
64
65         idx_cfg = self.config.get("indexing", {})
66         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
67         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
68
69         api_key_env_var = f"{provider_name.upper()}_API_KEY"
70         api_key = os.environ.get(api_key_env_var)
71         if not api_key:
72             logger.error(f"{api_key_env_var} not set; cannot run the {provider_name}
handoff.")
73         return []
74         try:
75             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
76             except KeyError as e:
77                 logger.error(str(e))
78                 return []
79
80             video_duration = get_video_duration(video_path)
81             if video_duration <= 0:
82                 logger.error("Invalid video duration.")
83                 return []
84             fps, frame_width = self._probe_fps_width(video_path)
85
86             wasb_cfg = WasbConfig.from_indexing_cfg(idx_cfg)
87             runner: DetectorRunner = WasbRunner(wasb_cfg)
88
89             w = idx_cfg.get("wasb", {})
90             velocity_thresh = w.get("velocity_threshold", 0.02)
91             merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
92             min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
93             handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
94             ai_max_workers = idx_cfg.get("ai_max_workers", 5)
95             log_candidates = idx_cfg.get("log_yolo_candidates", True)
96             store_candidates = idx_cfg.get("store_yolo_candidates", True)
97
98             detection_params = {
99                 "detector": "wasb_hybrid",
100                 "velocity_thresh": velocity_thresh,
101                 "merge_gap": merge_gap,
102                 "min_action_window_duration": min_window_duration,
103                 "weights_path": wasb_cfg.weights_path,
104             }
105
106             ok, msg = runner.healthcheck()
107             if not ok:
108                 logger.error("WASB WSL environment not ready: %s", msg)
109                 return []
110             logger.info("WASB WSL env OK: %s", msg)
111
112             # --- Phase 2: shuttle trajectory -> candidate rally windows ---
113             t_start = time.time()
114             out_dir = os.path.join("output", "wasb")
115             csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
116             if not csv_path:
117                 logger.error("WASB produced no trajectory; aborting.")
118                 return []
119
120             points = runner.parse_trajectory_csv(csv_path)
121             logger.info("Parsed %d trajectory points (%d visible).",
len(points), sum(1 for p in points if p.visible))
122

```

```

123         all_candidate_windows = runner.trajectory_to_action_windows(
124             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
125             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
126             min_window_duration=min_window_duration, chunk_index=0,
127         )
128     )
129     logger.info("Phase 2 (WASB) completed in %.1fs. Candidate windows: %d",
130                 time.time() - t_start, len(all_candidate_windows))
131
132     if log_candidates:
133         for cw in all_candidate_windows:
134             logger.info(f"[WASB rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} |
135                         f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
136                         f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
137
138         candidate_ids = [None] * len(all_candidate_windows)
139         if store_candidates:
140             for i, cw in enumerate(all_candidate_windows):
141                 stats = {
142                     "peak_velocity": cw["peak_velocity"], "mean_velocity":
cw["mean_velocity"],
143                     "active_frame_count": cw["active_frame_count"], "track_id_count":
cw["track_id_count"],
144                 }
145                 candidate_ids[i] = self.db.add_candidate_segment(
146                     video_id, detector="wasb_hybrid", start_time=cw["start"],
end_time=cw["end"],
147                     chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
148                     stats=stats, detection_params=detection_params,
149                 )
150
151         if not all_candidate_windows:
152             return []
153
154     # --- Phase 3: AI provider handoff (shared pattern) ---
155     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
156     os.makedirs(handoff_dir, exist_ok=True)
157     logger.info("Phase 3: %s validation on %d candidate windows...",
158                 provider_name, len(all_candidate_windows))
159
160     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
161         w_start = max(0, window["start"] - handoff_padding)
162         w_end = min(video_duration, window["end"] + handoff_padding)
163         w_dur = w_end - w_start
164         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
165         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
166             return []
167         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
168         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
label=f"Handoff {wi+1}")
169
170         valid = []
171         for seg in segments:
172             try:
173                 seg["start_time"] = float(seg["start_time"]) + w_start
174                 seg["end_time"] = float(seg["end_time"]) + w_start
175                 seg["_candidate_id"] = candidate_ids[wi]
176                 valid.append(seg)
177             except (ValueError, TypeError, KeyError) as e:
178                 logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
179         try:

```

```

180         os.remove(w_path)
181     except OSError:
182         pass
183     return valid
184
185     ai_segments: List[dict] = []
186     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
187         futures = [executor.submit(process_candidate_window, wi, cw)
188                     for wi, cw in enumerate(all_candidate_windows)]
189         for future in futures:
190             ai_segments.extend(future.result())
191     try:
192         os.rmdir(handoff_dir)
193     except OSError:
194         pass
195     logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
196
197     # --- Phase 4: DB population (shared pattern) ---
198     segments_data = []
199     for segment in ai_segments:
200         try:
201             start = float(segment["start_time"])
202             end = float(segment["end_time"])
203             dur = end - start
204             shot_count = segment.get("shuttle_exchange_count")
205             try:
206                 shot_count = int(shot_count)
207             except (ValueError, TypeError):
208                 shot_count = max(3, int(dur / 0.8))
209             metadata = {"shot_count": shot_count,
210                        "shot_count_confidence":
segment.get("shot_count_confidence", "Unknown"),
211                        "detector": f"wasb-hybrid-{model_name}"}
212             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
213             if segment_id:
214                 candidate_id = segment.get("_candidate_id")
215                 if candidate_id is not None:
216                     self.db.link_candidate_to_segment(candidate_id, segment_id)
217                 segments_data.append({"id": segment_id, "start_time": start,
"end_time": end,
218                                     "duration": dur, "metadata": metadata})
219         except Exception as e:
220             logger.warning("Skipping invalid segment struct: %s", e)
221
222     return segments_data
223
224
225     segmenter_registry.register(WasbHybridSegmenter)
226

```

8. user (2026-06-10T09:28:28.151Z)

```

1     # Owned-model implementation plan (roadmap) ? DRAFT for owner review
2
3     **Status:** PLAN / roadmap. No owned-model *implementation* starts until the owner
greenlights a specific
4     milestone. Paid Gemini stays the always-ready inference/fallback throughout (never a
training source).
5     **Grounded in:** `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md`, the n=6 LOVO +
learned prototype
6     (`docs/archives/quality-iterations/2026-06-multivideo-lovo/`), and a verified strategy
workflow (2026-06-09).

```

```

7
8     ## Design principle (owner directive): *envision now, build later*
9     Every deferred piece (conversational-QA, on-device, multisport, the VLM) must be
**architecturally envisioned
10     now** so we never hit a mammoth refactor. We **design the seams** (data contracts,
export boundaries, the
11     event store) up front and **implement behind them** when each milestone is greenlit.
Punting a *feature* is fine;
12     punting its *seam* is not.
13
14     ## North Star (reframed per PR #61 review)
15     - **Immediate deliverable:** a **training framework + harness** that produces a model
with **very high precision/
16     recall for extracting highlights + rallies from a clip**. *Quality is the first-order
objective.*
17     - **Eventual product:** a **single, sport-agnostic, conversational** video model ?
upload a clip, then ask
18     contextual questions ("longest rally", "make a clip of all service faults", "rallies
over 20 shots").
19     - **On-device is LATER** (a model-compression goal so power users save upload
cost/hassle) ? definitely on the
20     roadmap, but **not** the immediate target. Don't over-index the strategy on it now.
21     - **The moat = the consented dataset + the eval harness, not the weights.**
22
23     ## Hard guardrails (non-negotiable ? "never corrupted")
24     - **Commercial-clean licensing for EVERY dependency ? code, data, AND model weights: MIT
/ Apache-2.0 / BSD only.**
25     (Ratifies the prior "Apache-2.0 only" to include MIT/BSD.) Reject viral/NC/custom-
restrictive: Gemma 3 (viral),
26     **Gemma 3n** (Gemma Terms follow derivatives ? even cleaner reject), Llama 4 (700M-MAU
cap), Commons-Clause repos.
27     - **Never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs.** Also forbidden: public
grounding-CoT datasets that
28     were Gemini-generated (e.g. TVG-R1's 56K) ? we must originate our own CoT supervision
(a real, binding cost).
29     - **Exclude minors; per-user training data needs a separate explicit opt-in; split data
by match.**
30     - ? **Base-model pretraining provenance is unauditabile for *every* open VLM (incl.
Qwen3-VL).** Our "no paid-LLM
31     training" rule is enforceable at *our fine-tuning data* layer, not the base's
pretraining ? an **explicit owner
32     risk-acceptance** is required to use any open VLM. (Governed by the base's license,
not our pipeline.)
33
34     ## The model strategy (verified 2026-06-09)
35     **Framework ? base** (the Gemma-4-vs-ms-swift clarification): a *framework* is the forge
(runs SFT/GRPO, emits
36     weights); a *base* is the metal you forge. You pick one framework and feed it any clean
base.
37     - **Framework: ms-swift (Apache-2.0)** ? the one forge that does native **video + audio
+ timestamp-grounding +
38     GRPO/RFT with a custom video reward** in one stack. Keep **unsloth** (Apache-2.0 core;
*avoid its AGPL Studio UI*)
39     as the 8 GB-local SFT/compression tool for the deferred on-device goal. ? Point-in-
time snapshot ? re-verify at impl.
40     - **Base (when we reach the VLM): Qwen3-VL (Apache-2.0, uniform across sizes)** ? long-
video context, purpose-built
41     **timestamp emission** (second-level localization), multi-turn video chat. Re-confirm
the exact size's LICENSE
42     pre-release. **Gemma-4 stays AMBER and bench-only** ? its Apache grant is likely but
its Prohibited-Use posture is
43     unconfirmed (counsel needed), *and* its video/timestamp capability is disputed; not
the primary extractor.
44     **InternVL3 (MIT/Apache)** = clean fallback if a Qwen blocker appears. ? base swap is
cheap *only within

```

```

45     timestamp-capable Apache bases* (timestamp data-format + grounding-CoT are base-
specific).
46     - **START WITH THE TAL HEAD, NOT THE VLM** (the decisive call): fine-tuned VLMs **miss
strict temporal boundaries**
47     (best RL-post-trained video VLM ? R@0.7 50% on open-domain benchmarks; "coarse
regions, not accurate boundaries").
48     **Boundary accuracy IS the product.** The head (`rally_seq_proto.py`) is already de-
risked on our corpus (held-out
49     AUC 0.83?0.92), trains in minutes at ~4.5 GB on the 2070 Super ($0), is ~1M params
(trivially compressible later),
50     and **doubles as a pseudo-label teacher + the honest baseline the VLM must beat.** So:
**head now ? VLM later**,
51     strictly sequenced. When the VLM comes, prefer **two-stage** (VLM coarse-proposes,
head refines boundaries) over
52     wholesale replacement. *(Open: the VLM-ceiling number is open-domain; our amateur
footage may differ ? re-measure.)*
53
54     ## Conversational video-QA ? build the bookkeeping NOW, punt the model
55     **Punt the conversational *feature*** until extraction quality is solid (current LOVO
~0.583 ? "very high"; a chat UI
56     over a weak extractor exposes the weakness). **Reject end-to-end-VLM-answers-each-turn**
(expensive; bad at exact
57     counting; the strong teachers are license-forbidden). **But build the *seam* now** (the
brief requires it):
58     - **Architecture = "extract once, query many":** EXTRACTION (TAL head ? later VLM) emits
structured timestamped
59     spans ? a durable **per-video EVENT STORE** ? a **structured QUERY layer** (text-to-
SQL + ffmpeg clip-cut). The
60     cited queries ("longest rally", "all service faults", "rallies > 20 shots") are
**structured-numeric ? SQL, not a
61     VLM job** (SQL counts exactly where VLMs hallucinate).
62     - **Designed-now seam:** make the **per-video event-index schema a first-class data
contract** alongside the M0.1
63     corpus spine: per rally `{video_id(MD5), rally_ordinal, sport, start, end, shot_count,
ending_reason/fault_tags,
64     derived_clip_ref, provenance, confidence}`. **Reserve the event/fault-tag taxonomy
(e.g. `service_fault`) up front**
65     even if unpopulated. ? This is a real DB delta (a `PRAGMA user_version` migration):
`play_segments` today has
66     `video_id/start_time/end_time/duration/server/receiver/metadata` only; `highlights`
isn't a table yet ? so it's an
67     *extension with new columns + a highlights table*, not a no-op. **Punt** the NL
parser/agent until quality clears the bar.
68     - Paid Gemini may answer genuinely open-ended edge questions **at inference** over our
structured index (allowed ?
69     it reasons over our data, it is not trained on its outputs).
70
71     ## Multisport (single sport-agnostic model)
72     - **Model/framework: UNCHANGED.** One shared TAL head + one Qwen3-VL base, fine-tuned
across sports (sport = an input
73     feature / per-sport calibration). The collector + indexer are already sport-generic. ?
**ACTION: rename the repo off
74     "badminton" ASAP** (owner emphatic) ? consistent with this.
75     - **What multisport changes = DATA + DETECTORS (scale up, the accepted "good
problem"):** each new sport needs its own
76     labeled matches (split by match) + a **clean trajectory detector** to feed the head's
features. **The binding blocker
77     is per-sport detectors:** WASB (badminton) has the unresolved **C3** checkpoint-
provenance issue, and **clean MIT/
78     Apache/BSD shuttle/ball detectors for tennis/TT/pickleball/padel are not yet
identified.** Sequence by public-data
79     richness: **badminton ? tennis/TT ? pickleball/padel.**
80     - ? **Single-model-multisport-temporal-generalization is an OPEN HYPOTHESIS**, not
proven ? our corpus is badminton-only,
81     and the evidence once cited for it (RacketVision) was *refuted on verification* (it's

```

```

ball-tracking, not temporal). The
82     concept is sound on first principles (rally signal = a racquet-sport invariant) but
must be validated once a 2nd-sport
83     corpus exists. Don't cite RacketVision as evidence.
84
85     ## Language ? Python everywhere; on-device runtime is a deferred, layer-local export
shell
86     - **Training/eval = Python-locked, and that's fine** (PyTorch/ms-swift/ActionFormer);
offline/batch, no latency hot path.
87     - **Platform = no Python hot path to escape** ? the heavy bytes/GPU already run out-of-
process (ffmpeg, WASB-in-WSL, cloud
88     GPU per-job); the control plane is I/O-bound. **Reject a Go/Rust hot-path split** ?
single-digit-ms wins dwarfed by
89     ~40-min GPU jobs, and it would fracture the one codebase holding the QA-state + eval
harness + provider registry.
90     - **On-device (deferred) = the only place a 2nd language appears, and it doesn't dictate
our dev language.** train in
91     Python ? **export across the boundary** (ONNX/Core ML/ExecuTorch/GGUF) ? a thin device
shell (Swift/Kotlin/C++/Rust)
92     runs the artifact with zero Python. **The one concrete constraint now: train with
export-clean ops** so the compression
93     target stays reachable without re-architecting. (On-device base stays on the
Qwen3-VL-4B Apache lineage ? *not* Gemma-3-270M.)
94     - Escalation order if a real Python CPU hot path ever appears: free-threading ?
numpy/vectorize ? Cython/PyO3 ? only then a
95     separate service. **None exists today.**
96
97     ---
98
99     ## Phase 0 ? Foundation (start now, per-milestone greenlight; $0, local, no cloud/DPA)
100
101     ### M0.1 ? The labeled-corpus spine **+ the event-index contract** (the moat) .
*greenlight item*
102     Canonical JSONL row per rally (corpus) **and** the queryable per-video event-index
schema (the conversational seam)
103     ? same row, wired to a store. Fields: `{video_id(MD5), rally_ordinal, sport, start, end,
ending_reason/fault_tags,
104     shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY
MATCH), trajectory_ref,
105     labeled_window, derived_clip_ref, confidence}`. 3 transcoders (heuristic/LogReg . TAL
ActivityNet-JSON . VLM clip?QA),
106     one scorer spine (`metrics.match_intervals`). Authored in `sports-data-collector`
(system-of-record; extends PR #13).
107     **Reserve fault/event taxonomy now.** Acceptance: round-trip test; split-by-match
enforced; **no Gemini-sourced rows in any training view**.
108
109     ### M0.2 ? Grow the corpus . **DONE (n=6), running** . *owner action + my scoring loop*
110     6 distinct matches labeled (3 singles incl. Mahadevpura, 1 doubles, 2 multi-court); per-
match + LOVO recorded; the
111     "contrast" data-quality metric validated. Owner adding 1 more varied multi-court-club
match ? margin. **Bottleneck =
112     data + calibration, not method.** Keep growing per new sport.
113
114     ### M0.3 ? Compliance cleanup (**hard blocker ? do before any training run**) .
*greenlight item*
115     - **(i) PURGE the Gemini-silver TRAINING path** ? the live no-paid-LLM-training
violation is
116     **`backend/eval/distill_local.py`** (it trains a classifier on Gemini-silver CSV
labels ? `output/local_rally_model.json`)
117     and the threshold distillation in `calibrate_local.py`. (NB:
`backend/eval/training.py::label_candidates` is a *pure,
118     source-agnostic* function ? there is no store by that name; the earlier
"training.label_candidates" pointer was wrong.)
119     Nothing Gemini-trained is in git (artifacts live under gitignored `output/`), but the
path that regenerates them is one

```

```

120     command away ? retarget to human + open-teacher (own-model/WASB) labels; Gemini =
eval/reference/fallback only.
121     - **(ii) WASB C3 ? first-class action item (owner):** the academic-data checkpoint
provenance is a **commercial-ship
122         blocker** that a clean head does NOT launder. Resolve via **own-trained weights** or a
clean MIT/Apache detector swap.
123         ? **own-weights is an un-scoped NEW workstream, not ROADMAP Phase 4:** per ADR-002 our
temporal labels can't train the
124         shuttle detector (it needs per-frame coordinate labels we don't have). **Multiplies
per sport** (incl. any TT-on-WASB use).
125         Carry as a tracked blocker that **gates SHIP** (not Gen-0 research).
126
127     ### M0.4 ? Gen-0 TAL harness (**FIRST-CLASS ACTION ITEM** per owner) . *greenlight item*
128     Productionize `rally_seq_proto.py` ? a **one-command train?eval?ship-gate** harness over
**ActionFormer (MIT, 1:1
129     rally=instance)** and/or **ASFormer (MIT, TAS)** (use `sj-li/MS-TCN2` if MS-TCN++, *not*
the Commons-Clause repo),
130     reusing `metrics.match_intervals`. **Define the ship-gate target up front** (open item):
a FLOOR of "beat the honest n=6
131     heuristic LOVO **0.480**" is necessary but not sufficient ? set an explicit **F1@tIoU**
target for "very high P/R" (e.g.
132     F1@tIoU=0.5 for highlights, tighter for precise rally cuts) before declaring victory.
Trains on the GPU/WSL box.
133
134     ---
135
136     ## Phase 1 ? Gen-0 ship decision (gated on n?5 + M0.4)
137     Run the A/B; if the learned Gen-0 clears the ship-gate with per-video calibration, it
becomes the rally/highlight
138     extractor (local, MIT). If not, the gap (more data/features) is the next signal ? no
overfitting to declare victory.
139
140     ## Phase 2 ? Gen-1 / Gen-2 (gated on platform scaffolding P0?P4 + healthy corpus +
explicit go)
141     - **Gen-1:** concatenate fully-local cue channels onto the head. ? **Audio is NOT a
promising cue here** ? our E1 probe
142     (PR #35) found it weak (~2.2x discrimination, AUC ~0.6) and the foreground-audio idea
also failed this session; keep it
143     parked, not "in reserve." Pose/court is the more credible #2 cue (license/cost-vet
first).
144     - **Gen-2 (VLM):** ms-swift fine-tune of **Qwen3-VL** (SFT cold-start ? GRPO/RFT with a
temporal-IoU-vs-golden reward,
145     our own CoT supervision). **Two-stage** (VLM proposes, head refines). Cloud train on
Modal (no-access) / RunPod-Secure.
146     - **Hard pre-serving gates (unmeasured, load-bearing):** the **$/call crossover** (self-
hosted VLM vs Gemini) **+ a real
147     VRAM/token + cost benchmark** on rented A100/L40S (ms-swift multimodal-GRPO examples
use 6?8 GPUs ? **budget is the
148     largest open risk and is currently unquantified**). Don't stand up an owned serving
stack until these clear.
149
150     ---
151
152     ## Decisions / status
153     | # | item | status |
154     |---|---|---|
155     | 1 | Greenlight Phase-0 milestone(s) ? rec: **M0.1 corpus+event-index**, then **M0.4
harness** | awaiting owner |
156     | 2 | WASB C3 ? first-class action item (own-weights / clean swap) | **owner: yes** ? |
157     | 3 | Ratify licensing **MIT / Apache-2.0 / BSD** (code+data+weights) | **owner: yes** ?
|
158     | 4 | Collector = corpus + event-store system-of-record | **owner: confirm?** |
159     | 5 | Rename repo off "badminton" | **owner action** ? |
160     | 6 | Conversational-QA: build seam now, punt model | **rec accepted?** |
161

```

```

162     ## Open risks / unknowns (none block Phase-0 research)
163     Budget/cost to reach Gen-2 **unquantified** (largest risk) · ship-gate "very high P/R"
target undefined · per-sport clean
164     trajectory detectors not yet identified · base-model pretraining provenance unauditale
(risk-acceptance) · real VLM
165     VRAM/token cost · single-model-multisport-temporal generalization unproven · DPA + no-
train (minors-excluded) for cloud
166     training · on-device 4B latency on 8 GB unmeasured.
167
168     ---
169     *Design-only and reversible. Nothing starts until you greenlight a milestone. Sequencing
discipline holds: the committed
170     next platform slice remains the async job model (single-tenant); only the M0.1?M0.4
foundation is safe to start.*
171

```

9. assistant (2026-06-10T09:28:36.256Z)

Let me look at the eval stack layout, the WASB runner's external-repo consumption pattern, and the collector side to ground the precedent claims.

10. user (2026-06-10T09:28:37.657Z)

```

backend\eval\__init__.py
backend\eval\annotations.py
backend\eval\classifier.py
backend\eval\training.py
backend\eval\calibration.py
backend\eval\gemini_refine.py
backend\eval\calibrate_local.py
backend\eval\calibrate_wasb.py
backend\eval\distill_local.py
backend\eval\export_wasb_segments.py
backend\eval\harness.py
backend\eval\batch.py
backend\eval\eval_partial_labels.py
backend\eval\metrics.py
backend\eval\rally_seq_proto.py
backend\eval\regression.py

```

11. user (2026-06-10T09:28:38.772Z)

```

6: 3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
34: repo_dir: str = "~/models/WASB-SBDT"
35: conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36: conda_env: str = "wasb"
37: weights_path: str = "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
40: timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
47:     repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
48:     conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
49:     conda_env=w.get("conda_env", "wasb"),
50:     weights_path=w.get("weights_path",
51:         "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
63:     full = f"source {self.cfg.conda_sh} && {inner_cmd}"
71:     cmd = (f"conda activate {self.cfg.conda_env} && "
133:         f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
137:         f"--weights {shlex.quote(self.cfg.weights_path)} "

```

12. user (2026-06-10T09:28:39.239Z)

```

C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tennis.py
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\cricket.py
C:\Users\avidu\Projects\sports-data-collector\src\parsers\base_parser.py

```

C:\Users\avidu\Projects\sports-data-collector\src\core\schemas__pycache__\tennis.cpython-313.pyc
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas__pycache__\cricket.cpython-313.pyc
C:\Users\avidu\Projects\sports-data-collector\src\parsers__pycache__\base_parser.cpython-313.pyc
C:\Users\avidu\Projects\sports-data-collector\pytest_cache\gitignore
C:\Users\avidu\Projects\sports-data-collector\pytest_cache\README.md
C:\Users\avidu\Projects\sports-data-collector\pytest_cache\CACHEDIR.TAG
C:\Users\avidu\Projects\sports-data-collector\data\mock_local_badminton_anno.json
C:\Users\avidu\Projects\sports-data-collector\data\staging_coachai_match.csv
C:\Users\avidu\Projects\sports-data-collector\data\temp\dummy_cleanup_test.csv
C:\Users\avidu\Projects\sports-data-collector\.git\description
C:\Users\avidu\Projects\sports-data-collector\.git\hooks\applypatch-msg.sample
C:\Users\avidu\Projects\sports-data-collector\.git\hooks\commit-msg.sample
C:\Users\avidu\Projects\sports-data-collector\.git\hooks\fsmonitor-watchman.sample
C:\Users\avidu\Projects\sports-data-collector\.git\hooks\post-update.sample
C:\Users\avidu\Projects\sports-data-collector\.git\hooks\pre-applypatch.sample
C:\Users\avidu\Projects\sports-data-collector\.git\hooks\pre-commit.sample
C:\Users\avidu\Projects\sports-data-collector\.git\hooks\pre-merge-commit.sample
C:\Users\avidu\Projects\sports-data-collector\.git\hooks\pre-push.sample
C:\Users\avidu\Projects\sports-data-collector\.git\hooks\pre-rebase.sample
C:\Users\avidu\Projects\sports-data-collector\.git\hooks\pre-receive.sample
C:\Users\avidu\Projects\sports-data-collector\.git\hooks\prepare-commit-msg.sample
C:\Users\avidu\Projects\sports-data-collector\.git\hooks\push-to-checkout.sample
C:\Users\avidu\Projects\sports-data-collector\.git\hooks\sendemail-validate.sample
C:\Users\avidu\Projects\sports-data-collector\.git\hooks\update.sample
C:\Users\avidu\Projects\sports-data-collector\.git\info\exclude
C:\Users\avidu\Projects\sports-data-collector\.git\objects\58\d03999b0be9122f73925d461cc3230a97dc781
C:\Users\avidu\Projects\sports-data-collector\.git\objects\9f\f13b91c50c0c9a2c20e59d2346b0160b0529cb
C:\Users\avidu\Projects\sports-data-collector\.git\objects\b0\549e0acc1885ccb039609385703c658882afe
C:\Users\avidu\Projects\sports-data-collector\.git\objects\ca\fffd031463973f3fea4c2500b74a9c48eec19a
C:\Users\avidu\Projects\sports-data-collector\.git\objects\fb\8c6db4d43411a3c37276c82fbee9582b1e8ea8
C:\Users\avidu\Projects\sports-data-collector\.git\objects\ce\b4223211e5e5cff5b23e2a5ba60fc53d6328fd
C:\Users\avidu\Projects\sports-data-collector\.git\objects\0f\5ead84808ce318abb51dd9db4d9cbd3373a3f3
C:\Users\avidu\Projects\sports-data-collector\.git\objects\27\77fed31486d17c14b0e8a93f725b1ba2b657ab
C:\Users\avidu\Projects\sports-data-collector\.git\objects\82\5cbd61996b29e1ab2c3d80034c5e385c83844c
C:\Users\avidu\Projects\sports-data-collector\.git\objects\2d\1f015725c4069a997040928ee1d2b6e581890e
C:\Users\avidu\Projects\sports-data-collector\.git\objects\3e\7c3c2047108870d0346426b5d489655f1e09e3
C:\Users\avidu\Projects\sports-data-collector\.git\objects\4b\ff82369b7a6df0730b917b5a6752638de56a32
C:\Users\avidu\Projects\sports-data-collector\.git\objects\98\1a14e908a0e112e545300094b3baa4f0d8d2ec7
C:\Users\avidu\Projects\sports-data-collector\.git\objects\c1\336d00ca33611958c3fc8591e4f9ba32c22a3c
C:\Users\avidu\Projects\sports-data-collector\.git\objects\ee\14b8b2ff4b40f1c537410872aed4551e557168
C:\Users\avidu\Projects\sports-data-collector\.git\objects\05\ca49e61c0d193cb16a8546e1d5f44360b88649
C:\Users\avidu\Projects\sports-data-collector\.git\objects\25\bcb9035497b5de2a1de8857ffc7b869d28935
C:\Users\avidu\Projects\sports-data-collector\.git\objects\29\ef9fe3490a8f8746bd6ac230e83df97fd24513
C:\Users\avidu\Projects\sports-data-

collector\.git\objects\34\6e5191fdf56019daa8b18fb51cfb337862562a
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\b0\c7a97572e7af41ba7949d34bf13fdd13116371
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\c1\bb076f4ffe5b48a644e29d9d9e2539ee9572e9
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\d4\6625fe0b9bd8dd72995008ec1178df87267568
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\ec\23c51df24d77f3b125845438b2af50fe60d303
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\f2\4b649ee046da206c9f1b79d5cf4a10c64d8857
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\3b\1b0cf442368e06845a199214a83a48f91304ca
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\ef\bd82efeedb9dac6f5a1ee1f78d0aabe333534b
C:\Users\avidu\Projects\sports-data-collector\.gitattributes
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\70\715c4db95ff9056b33dfa0e937d9ea57446801
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\19\781ada6a6f58499ffb1df46b0511fdc4417e2b
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\95\e78c1b2ef3d24a0c80dc7005ff582718f5daf7
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\7a\c279cffbafeb42613fbc7376f05790af7dc278
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\be\6d7ca17c2f760986d14d7dbe896e51c3c2ec84
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\cd\533f4e5b41de3d13d5167397483fdde20304eb
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\d7\37c9d6fab0b7ae810c1c1d122cbea277b8a682
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\dd\27711332765159f57ea6829cb4722865679c24
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\e0\82cb430504eaf2c16f30c40baab3c812aad76f
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\66\1a0c94eaf50f9ca97ae01e0f0257d30c199ad4
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\2a\cb2f0241f5a45d8e481ac0824a676cc20ffc11
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\6b\2edd83f7208716eff6f2c81c496df49b0828cd
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\7a\fe94ac36ef00b434f9aa910ee0601280a4b30b
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\85\4b07772caad7b322709e55f883c584036b6c6a
C:\Users\avidu\Projects\sports-data-collector\config\config.yaml
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\12\0178cfd22c939b9c6c89c9fc8e0751178650f
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\1a\8da6c5a912c91ad860c8c1d6f92c379ba37ec1
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\3f\946a2065e54ef10befd0e3578656bd8fc18dc9
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\ac\6d8a40b008fff586a22ae276a8d47619ec1711
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\9e\2907a755eaf7b115fb4f1f04832add8d8607ac
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\ec\69683c4884f013f2afb875c47d6d96d587867a
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\0a\3983d0e0774dec8c548cf1185672d87b994788
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\04\c22e10566d81427fdae9ccfef0e62384459311
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\1b\29dfc7f18678a8632d3415529857d8bdf437b5
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\5c\b2abc9d212392a3afadd5eb8ca6acb4b5615c8

```

C:\Users\avidu\Projects\sports-data-
collector\.git\objects\5e\57bba18b3be8e72a9a7a9eb7e9304ccfb46af5
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\69\f79c4367cf8e1d3f7b4702bbfd6c0801d95625
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\6a\73d620c53e0a519fac384439b3f7cde70ddd0f
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\6d\d0d63b1d81acf45891887c3b23750753a5219d
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\58\17c9456bf51773de0a2adf9a884653737e1a51
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\7d\8bb10fb36faa79298ebc9e173501c1999fa8ea
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\99\9e6055beaccb8dc26e9987e3a2c51a623de18
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\97\4590e4f20daf8f7aea4d035edf7f9fcc7ca363
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\1f\96dd7b4b3970c7cc9fff6d1c55b7b939458ca3
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\7d\31b689c0c3d93ad52b22745c588a461f0c2e4a
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\09\90923c1b823081e0cd8809356a7660ff3d6197
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\3f\9493f5fed0c5f6fc9117b6641992cb674f283
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\64\aa2fea7c14226bb4b0ae13e7e7936516758c24
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\8e\7bbdf70e3606b60f48e77dd4cdd6d05f8346f7
C:\Users\avidu\Projects\sports-data-
collector\.git\objects\b0\01cbd47ddfc3326c62ccaff19bf3501649a96c
C:\Users\avidu\Projects\sports-data-collector\data\videos\_diag.mp4.part
C:\Users\avidu\Projects\sports-data-collector\data\videos\_diag.mp4.ytdl
C:\Users\avidu\Projects\sports-data-collector\data\videos\_diag.mp4.part-Frag320.part
C:\Users\avidu\Projects\sports-data-collector\data\videos\_t.mp4.part
C:\Users\avidu\Projects\sports-data-collector\data\videos\_t.mp4.ytdl
(Results are truncated. Consider using a more specific path or pattern.)

```

13. user (2026-06-10T09:28:48.968Z)

```

35-## Prioritized next steps
36-1. **[owner greenlight] Phase-0 (M0.x), $0\local:**
37: - **M0.4 ? Gen-0 TAL harness** (first-class): productionize `rally_seq_proto.py` ? one-
command train?eval?ship-gate** over
38-   ActionFormer/ASFormer, reusing `metrics.match_intervals`. **Define the ship-gate
target** (explicit F1@tIoU; floor = beat
39-   heuristic LOVO **0.480**; the corrected learned prototype already sits at **0.626**) ?
"very high P/R" must be
40-   operationalized, not just "beat the floor." Add **paired per-video stats**
(sign/Wilcoxon) ? at n=6 a mean gap alone is noise-ambiguous.
41: - **M0.1 ? corpus spine + the per-video event-index contract** (the conversational seam:
extract ? event store ? text-to-SQL).
42-   Includes importing the n=6 golden matches into the collector (now safe: import guard +
`--normalize`, PR #15).
43- - **M0.3 ? compliance:** ? corpus licensing reclass DONE (collector #14/#17 ? ShuttleSet
demoted, export owned-only).
--
57- - **Gemini-wrapper baseline vs the n=6 LOVO corpus** (harness exists) ? if a wrapper ?
learned 0.626, the owned-model
58-   investment is premature and the play is distribution/UX.
59:4. **Housekeeping:** **rename repo off "badminton"** (multisport, owner action ? now in
BACKLOG); confirm collector =
60-   corpus/event-store system-of-record.
61-

```

14. user (2026-06-10T09:28:50.037Z)

```

45- unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives
(misses)
46-
47-
48: def match_intervals(preds: List[Interval], gts: List[Interval],
49-                     iou_threshold: float = 0.5) -> MatchResult:
50-     """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51-
52-     """
101-     return self.__dict__.copy()
102-
103-
104: def score(preds: List[Interval], gts: List[Interval],
105-           iou_threshold: float = 0.5,
106-           match_result: Optional[MatchResult] = None) -> Score:
107-     """Compute precision/recall/F1, mean matched IoU, and boundary errors.

```

15. user (2026-06-10T09:28:59.225Z)

No files found

16. user (2026-06-10T09:29:00.155Z)

```

1: """Batch evaluation across a collector export manifest.
3: Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
4: sports-data-collector's `curate export`, (optionally) process each video through
23:     """Resolve a manifest `local_path` to an absolute path.
25:     The collector stores paths like ``./data/videos\\shuttle1.mp4`` relative
28:     collector root). Returns None for a missing/empty path.
37:     # This keeps collector manifest paths working in tests and mixed environments.

```

17. user (2026-06-10T09:29:07.310Z)

No files found

18. user (2026-06-10T09:29:23.652Z)

```

10:     baseline      ? a per-video snapshot of "known good" numbers [this module]
13: baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
32: # actually load a committed baseline to compare against.
33: DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
38:     """Stable short hash of the GT intervals ? detects a baseline taken against different
labels."""
51:     # Baseline values compared against (None when no baseline existed yet).
52:     baseline_precision: Optional[float]
53:     baseline_recall: Optional[float]
57:     # True only when a baseline existed to compare against. Distinguishes a genuine PASS
59:     has_baseline: bool = False
66: # ----- baseline store
68: def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69:     """Load the per-video baseline snapshot file (returns {} if absent)."""
76: def _baseline_key(video_id: str, stage: str) -> str:
80: def save_baseline(video_id: str, stage: str, precision: float, recall: float,

```

[Showing results with pagination = limit: 15]

19. user (2026-06-10T09:32:03.885Z)

Structured output provided successfully

20. assistant (2026-06-10T09:32:13.889Z)

Position delivered via structured output. The case for a separate model/training repo is grounded in the indexer's own architecture: the DetectorRunner ABC

(`backend/pipeline/detectors/base.py`) already documents the external-substrate consumption pattern, WASB proves it works (external repo + weights path config in `wasb\_runner.py`), and the import-smoke CI job's no-GPU-stack guarantee (`.github/workflows/ci.yml`) is irreconcilable with ActionFormer/ms-swift dependencies living in-repo. The honest counterweight ? the observed collector/indexer header fork ? is addressed head-on: the corpus crosses a repo boundary regardless of where training lives, so the fix is the versioned artifact manifest + shared runtime package, not co-location.